RESTAURANT MANAGEMENT SYSTEM

COMPLETE PRODUCTION SPECIFICATION

Platform Type	Multi-Tenant SaaS
Frontend	Flutter (Web + Android + iOS)
Backend	Node.js + Fastify + PostgreSQL + Redis
Languages	French (fr) + Arabic (ar) — Full RTL
Target	**Production — Not MVP, Not Prototype**

1. Project Identity & Absolute Rules

1.1 What This System Is

This is a production-grade, multi-tenant SaaS Restaurant Management System. Every architectural decision, every line of code, every UI component must be built with production quality: fault-tolerant, secure, performant, maintainable, and scalable from day one. There is no MVP mode. There is no prototype mode. All features described in this document are in scope for the final shipped product.

1.2 Non-Negotiable Rules

✦ **RULE:** No tip management anywhere in the system. No tip field, no tip calculation, no tip UI, no tip database column. Completely absent.

✦ **RULE:** Staff accounts are created exclusively by the restaurant Admin. Staff cannot self-register, cannot sign up, cannot create their own accounts under any circumstance.

✦ **RULE:** Every tenant (restaurant) is fully data-isolated. No cross-tenant data access is ever possible.

✦ **RULE:** The Super Admin (platform owner) is the only entity that can create Admin-level restaurant accounts.

✦ **RULE:** The entire application UI, all labels, error messages, notifications, receipts, and PDFs must be available in French and Arabic with complete RTL layout support for Arabic.

✦ **RULE:** Node.js is the backend runtime. Fastify is the HTTP framework. No exceptions.

✦ **RULE:** All API responses on POS-critical paths must be under 200ms. All WebSocket events must propagate in under 500ms.

✦ **RULE:** The system must pass production security standards: HTTPS-only, JWT with rotation, RBAC on every endpoint, full audit logging, SQL injection prevention, input sanitization everywhere.

2. System Architecture

2.1 Overall Architecture Pattern

The system follows a three-layer architecture: a Flutter-based frontend (POS terminal app, back-office web app, mobile manager app), a Node.js/Fastify REST + WebSocket API layer, and a PostgreSQL/Redis/S3 data layer. All layers are containerized and horizontally scalable.

- Frontend → Backend: REST API (authenticated via JWT Bearer tokens) for all CRUD operations
- Frontend → Backend: WebSocket (Socket.io) for all real-time events: order updates, KDS tickets, table status, stock alerts
- Backend → Database: PostgreSQL via `pg` connection pool (PgBouncer in front for connection management)
- Backend → Cache: Redis for sessions, menu snapshots, rate-limit counters, real-time pub/sub relay, BullMQ job queues
- Backend → Storage: S3-compatible object store (Cloudflare R2) for item photos, receipts, supplier documents, staff files
- Backend → External: Payment gateway, SMS provider, email provider, aggregator webhooks

---

2.2 Multi-Tenancy Architecture

Each restaurant is an independent tenant identified by a UUID tenant_id. Tenant isolation is enforced at three levels:

- Database level: every table that contains tenant data has a tenant_id column, indexed, and all queries include WHERE tenant_id = $1. PostgreSQL Row Level Security (RLS) policies are enabled as a secondary defense.
- API level: every authenticated request extracts tenant_id from the JWT claims. Middleware validates that the resource being accessed belongs to the requesting tenant before executing any database query.
- Storage level: all files are stored under a tenant-scoped S3 prefix: /tenants/{tenant_id}/...

⚠ *A bug that allows cross-tenant data read is treated as a critical security incident, not a bug.*

2.3 Service Decomposition

The backend is organized as modular Fastify plugins (not separate microservices — monolith-first for operational simplicity, but with clean internal boundaries for future extraction):

Service Module	Responsibility	Key Dependencies
auth-service	JWT issuance, refresh, revocation, PIN login	Redis, bcrypt
tenant-service	Tenant CRUD, subscription plan, config	PostgreSQL
menu-service	Menu, categories, items, modifiers, 86	PostgreSQL, Redis cache
order-service	Order lifecycle, table state, POS flow	PostgreSQL, Socket.io, Redis

Service Module	Responsibility	Key Dependencies
kitchen-service	KDS events, cook times, course firing	Socket.io, PostgreSQL
inventory-service	Stock, BOM deduction, POs, suppliers	PostgreSQL, BullMQ
staff-service	Accounts, roles, attendance, scheduling	PostgreSQL, BullMQ
payment-service	Payment processing, receipts, refunds	Payment gateway SDK
reservation-service	Bookings, availability, reminders	PostgreSQL, BullMQ
crm-service	Customer profiles, loyalty, campaigns	PostgreSQL, BullMQ
analytics-service	Reports, forecasting, dashboard KPIs	PostgreSQL, Redis
notification-service	In-app, push, SMS, email dispatch	BullMQ, Twilio, SendGrid
finance-service	Cash reconciliation, P&L, tax, exports	PostgreSQL
super-admin-service	Platform tenants, billing, SaaS config	PostgreSQL, Stripe

3. Complete Technology Stack

3.1 Backend Stack

Runtime	Node.js 20 LTS
HTTP Framework	Fastify 4.x — chosen for ~80k req/s throughput, schema-based validation, plugin architecture
API Style	RESTful JSON API + WebSocket (Socket.io 4.x) on same server
ORM / Query Builder	Knex.js for migrations and query building; raw pg driver for performance-critical queries
Database	PostgreSQL 16 — primary data store for all transactional and relational data
Cache / Session	Redis 7 (Upstash or self-hosted) — sessions, menu cache, rate limits, pub/sub, job queues
Job Queue	BullMQ on Redis — async jobs: PDF generation, email/SMS, report exports, bulk operations
Authentication	JWT (jsonwebtoken) — access token 15min, refresh token 7 days, stored in Redis with revocation
Password Hashing	bcrypt with cost factor 12
Input Validation	Fastify JSON Schema validation on every route; Zod for business logic validation
File Storage	AWS S3 SDK v3 pointing at Cloudflare R2 (S3-compatible, zero egress cost)
Email	SendGrid or Resend — transactional emails (receipts, confirmations, staff onboarding, reports)
SMS	Twilio — reservation reminders, low stock alerts, staff notifications
PDF Generation	Puppeteer (headless Chrome) for receipts and reports — runs in BullMQ worker, never on main thread
Logging	Pino (built into Fastify) — structured JSON logs, shipped to Loki or Datadog
Monitoring	Prometheus metrics endpoint + Grafana dashboards; Sentry for error tracking
Rate Limiting	@fastify/rate-limit — per-tenant and per-IP limits on all routes
CORS	@fastify/cors — allowlist-only origins per tenant domain
API Documentation	Fastify Swagger (@fastify/swagger) — auto-generated OpenAPI 3.0 spec from route schemas
Testing	Vitest + Supertest for unit and integration; k6 for load testing
Connection Pooling	PgBouncer in transaction mode — max 20 connections per backend instance
Containerization	Docker — multi-stage builds, minimal alpine images
Orchestration	Docker Compose (dev/staging); Kubernetes or Railway/Fly.io (production)
CI/CD	GitHub Actions — lint, test, build, push image, deploy on merge to main
Secrets Management	Environment variables via .env (dev); Doppler or AWS Secrets Manager (prod)

3.2 Frontend Stack

Framework	Flutter 3.x — single codebase for Android, iOS, and Web
State Management	Riverpod 2.x — reactive, testable, compile-safe
Navigation	GoRouter — declarative, deep-link ready, guard-aware
API Client	Dio with interceptors for JWT injection, token refresh, error handling, and retry logic
WebSocket Client	socket_io_client — persistent connection with auto-reconnect, event subscriptions
Local Storage (Offline)	Hive (fast NoSQL) for offline order queue, menu snapshot, pending syncs
Localization	flutter_localizations + intl package. ARB files for fr and ar. Locale switching at runtime.
RTL Support	Directionality widget wrapping entire app; all layouts use start/end not left/right; custom RTL-aware components for numeric input, date pickers, and receipt layout
Forms & Validation	reactive_forms — declarative form management with validators
UI Components	Custom design system (no off-the-shelf theme). Material 3 base, fully overridden tokens for brand.
Charts & Analytics	fl_chart — fully custom, matches design system, supports RTL axis labels
Printing	flutter_thermal_printer + ESC/POS protocol for receipt printers (Epson, Star)
Barcode Scanning	mobile_scanner — camera-based barcode/QR scan for inventory and menu items
PDF Export	pdf package (dart) for client-side PDF generation (receipts, invoices); server-side for complex reports
Push Notifications	Firebase Cloud Messaging (FCM) via flutter_local_notifications
Secure Storage	flutter_secure_storage — store JWT refresh tokens in OS keychain
Network Detection	connectivity_plus — monitor online/offline status to trigger sync queue
Image Handling	cached_network_image — lazy loading with disk cache for menu item photos
Testing	Flutter test + integration_test + Patrol for E2E; golden tests for UI components

4. Database Schema — Full Production Design

4.1 Schema Principles

- Every tenant-scoped table has tenant_id UUID NOT NULL with FK to tenants.id and a composite index on (tenant_id, id)
- All primary keys are UUIDs (gen_random_uuid()) — no sequential integers exposed externally
- All tables have created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() and updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
- Soft deletes via deleted_at TIMESTAMPTZ NULL — queries always include WHERE deleted_at IS NULL
- Text fields for user-facing content have _fr and _ar variants: e.g., name_fr TEXT, name_ar TEXT
- Money values stored as INTEGER (smallest currency unit, e.g., centimes) — never FLOAT or DECIMAL for currency
- All migrations versioned and reversible via Knex migration files

4.2 Core Tables

Platform Layer

```
tenants (id, name, slug, plan_tier, status, billing_email, country_code,
currency_code,
         timezone, subscription_start, subscription_end, max_branches,
         created_at, updated_at, deleted_at)
```

```
tenant_config (id, tenant_id, key, value_json, created_at, updated_at)
  -- stores per-tenant settings: default_locale, tax_rate, receipt_footer_fr/ar,
  -- table_turn_alert_minutes, low_stock_default_par, etc.
```

```
subscription_plans (id, name, max_branches, max_staff, features_json,
                    monthly_price_cents, annual_price_cents)
```

Identity & Access

```
users (id, tenant_id, email, phone, password_hash, first_name_fr, first_name_ar,
       last_name_fr, last_name_ar, role, pin_hash, preferred_locale,
       is_active, last_login_at, created_by_user_id, created_at, updated_at,
deleted_at)
```

```
roles (id, tenant_id, name, permissions_json, is_system_role, created_at, updated_at)
  -- permissions_json: array of permission strings e.g.
["orders.create","reports.view"]
```

```
refresh_tokens (id, user_id, tenant_id, token_hash, expires_at, revoked_at,
                device_info, ip_address, created_at)
```

```
audit_log (id, tenant_id, user_id, action, entity_type, entity_id,
           old_value_json, new_value_json, ip_address, user_agent, created_at)
```

Branch & Floor Layout

```
branches (id, tenant_id, name_fr, name_ar, address, phone, timezone,
          is_active, created_at, updated_at, deleted_at)
```

```
floor_sections (id, tenant_id, branch_id, name_fr, name_ar, display_order,
                created_at, updated_at, deleted_at)
```

```
tables (id, tenant_id, branch_id, section_id, table_number, seats,
        pos_x, pos_y, width, height, shape,
        status, -- ENUM: available, occupied, reserved, cleaning
        current_order_id, occupied_since,
        created_at, updated_at, deleted_at)
```

Menu

```
menu_categories (id, tenant_id, branch_id, parent_id, name_fr, name_ar,
                 photo_url, display_order, is_active, created_at, updated_at,
deleted_at)
```

```
menu_items (id, tenant_id, branch_id, category_id, name_fr, name_ar,
            description_fr, description_ar, photo_url, base_price_cents,
            calories, allergens_json, is_available, is_86d,
            prep_station, -- ENUM: grill, cold, prep, pastry, bar
            display_order, created_at, updated_at, deleted_at)
```

```
modifier_groups (id, tenant_id, name_fr, name_ar, selection_type,
                 -- ENUM: single_required, single_optional, multi
                 min_selections, max_selections, created_at, updated_at, deleted_at)
```

```
modifiers (id, tenant_id, group_id, name_fr, name_ar, price_delta_cents,
           is_active, display_order, created_at, updated_at, deleted_at)
```

```
item_modifier_groups (item_id, group_id, display_order)
```

```
menu_schedules (id, tenant_id, branch_id, name_fr, name_ar, days_of_week,
                start_time, end_time, is_active, created_at, updated_at)
  -- many-to-many with menu_categories: schedule_categories(schedule_id, category_id)
```

```
recipes (id, tenant_id, item_id, instructions_fr, instructions_ar, plating_photo_url,
         portion_weight_grams, version, created_by_user_id, created_at, updated_at)
```

```
recipe_ingredients (id, recipe_id, tenant_id, ingredient_id, quantity, unit)
```

Orders & Payments

```
orders (id, tenant_id, branch_id, table_id, order_type,
        -- ENUM: dine_in, takeaway, delivery
        status, -- ENUM: draft, sent, in_progress, ready, served, closed, voided,
refunded
        customer_id, waiter_id, covers_count,
        subtotal_cents, discount_cents, tax_cents, total_cents,
        notes, created_at, updated_at, closed_at, voided_at)
```

```
order_items (id, order_id, tenant_id, item_id, seat_number, quantity,
             unit_price_cents, total_price_cents, notes, allergen_alert,
             course_number, status,
             -- ENUM: pending, fired, in_progress, ready, served, voided
             fired_at, ready_at, served_at, created_at, updated_at)
```

```
order_item_modifiers (id, order_item_id, modifier_id, name_fr, name_ar,
                      price_delta_cents)
```

```
payments (id, order_id, tenant_id, method,
          -- ENUM: cash, card_nfc, card_chip, card_swipe, qr_pay, voucher, gift_card
          amount_cents, reference, gateway_transaction_id,
          status, -- ENUM: pending, captured, failed, refunded, voided
          cashier_id, terminal_id, processed_at, created_at, updated_at)
```

```
discounts_applied (id, order_id, tenant_id, discount_rule_id, code_used,
                   applied_by_user_id, discount_type, discount_value,
                   amount_cents, authorized_by_manager_id, created_at)
```

Inventory

```
ingredients (id, tenant_id, branch_id, name_fr, name_ar, unit, category,
             current_stock, par_level, reorder_quantity, cost_per_unit_cents,
             supplier_id, barcode, storage_location,
             created_at, updated_at, deleted_at)
```

```
stock_movements (id, tenant_id, ingredient_id, movement_type,
                 -- ENUM: purchase, sale_deduction, waste, adjustment, stock_count
                 quantity_delta, stock_after, reference_id, reference_type,
                 notes, performed_by_user_id, created_at)
```

```
suppliers (id, tenant_id, name, contact_name, email, phone, address,
           payment_terms, lead_days, rating, is_active,
           created_at, updated_at, deleted_at)
```

```
purchase_orders (id, tenant_id, branch_id, supplier_id, status,
                 -- ENUM: draft, pending_approval, approved, sent, partial, received,
cancelled
                 subtotal_cents, tax_cents, total_cents,
                 approved_by_user_id, sent_at, expected_delivery_date,
                 notes, created_by_user_id, created_at, updated_at)
```

```
purchase_order_lines (id, po_id, tenant_id, ingredient_id, quantity_ordered,
                      quantity_received, unit_price_cents, total_cents,
                      received_at, discrepancy_notes)
```

```
waste_logs (id, tenant_id, branch_id, ingredient_id, quantity, unit,
            waste_type, -- ENUM: spoilage, over_prep, plate_return, other
            cost_cents, notes, logged_by_user_id, created_at)
```

Staff & HR

```
shifts (id, tenant_id, branch_id, user_id, start_time, end_time,
```

```
        break_minutes, status, -- ENUM: scheduled, active, completed, no_show
        created_at, updated_at)
```

```
attendance (id, tenant_id, user_id, shift_id, clocked_in_at, clocked_out_at,
            clock_in_method, -- ENUM: pin, nfc, biometric
            ip_address, device_id, created_at)
```

```
leave_requests (id, tenant_id, user_id, leave_type,
                start_date, end_date, days_count, reason,
                status, -- ENUM: pending, approved, rejected, cancelled
                reviewed_by_user_id, review_notes, created_at, updated_at)
```

Reservations & CRM

```
reservations (id, tenant_id, branch_id, customer_id, table_id,
              party_size, date, time_slot, duration_minutes,
              status, -- ENUM: pending, confirmed, seated, completed, no_show,
cancelled
              deposit_amount_cents, deposit_paid, special_requests,
              confirmation_code, created_at, updated_at)
```

```
customers (id, tenant_id, first_name, last_name, email, phone, birthday,
           preferred_locale, dietary_notes, loyalty_tier_id,
           loyalty_points, lifetime_spend_cents, total_visits,
           last_visit_at, created_at, updated_at)
```

```
loyalty_tiers (id, tenant_id, name_fr, name_ar, min_points,
               discount_pct, perks_json, created_at, updated_at)
```

```
loyalty_transactions (id, tenant_id, customer_id, order_id, points_delta,
                      balance_after, transaction_type, description, created_at)
```

5. API Specification

5.1 API Design Standards

- Base URL: https://api.{tenant-slug}.rms.com/v1 OR https://api.rms.com/v1 with X-Tenant-ID header
- All responses follow: { success: boolean, data: T | null, error: { code, message_fr, message_ar } | null, meta: { page, limit, total } | null }
- HTTP status codes used correctly: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error
- All endpoints protected by verifyJWT middleware except /auth/login, /auth/refresh, /reservations/public/:tenant
- All endpoints enforce tenant isolation via verifyTenantAccess middleware
- All write endpoints validate RBAC permissions via requirePermission('entity.action') middleware
- Pagination: cursor-based for lists (after, limit), offset-based for reports
- All list endpoints support filter, sort, and search query parameters

5.2 Authentication Endpoints

Method + Path	Description	Auth Required
POST /auth/login	Email+password login. Returns access_token (15min) + refresh_token (7d, httpOnly cookie).	No
POST /auth/login/pin	PIN login for POS terminals. Returns short-lived access token.	No
POST /auth/refresh	Rotate refresh token. Invalidates old one in Redis.	No (cookie)
POST /auth/logout	Revoke refresh token from Redis.	Yes
POST /auth/setup-password	First-time password set via onboarding token.	No (OTP token)
POST /auth/change-password	Authenticated password change.	Yes
GET /auth/me	Return current user profile, role, and permissions.	Yes

5.3 Core Operational Endpoints (Selected)

Orders

- POST /orders — Create new order (dine_in, takeaway, delivery). Validates table availability. Emits order:created WebSocket event.
- GET /orders — List orders with filters: status, date, branch, waiter, order_type
- GET /orders/:id — Full order detail including items, modifiers, payments

- PATCH /orders/:id — Update order status. Emits order:updated WebSocket event to kitchen and FOH.
- POST /orders/:id/items — Add item(s) to existing open order. Triggers KDS update.
- PATCH /orders/:id/items/:itemId — Update item status (fire, void, serve)
- POST /orders/:id/fire-course — Fire a specific course number to kitchen
- POST /orders/:id/payments — Process payment. Handles split-payment accumulation. Closes order when total paid.
- POST /orders/:id/refund — Initiate refund. Requires manager_authorization_code.
- POST /orders/:id/void — Void entire order. Requires manager_authorization_code. Restores table status.

Menu

- GET /menu — Full menu tree for a branch. Cached in Redis per tenant+branch. Invalidated on any menu write.
- POST /menu/categories — Create category. Validates parent_id if sub-category.
- PUT /menu/items/:id — Update item. If is_available changes to false (86d), emits menu:item_86d WebSocket broadcast.
- GET /menu/items/:id/recipe — Fetch recipe with plating guide and BOM.

KDS

- GET /kds/tickets — Active tickets for a station. Filtered by prep_station query param.
- PATCH /kds/tickets/:orderId/items/:itemId/bump — Mark item as ready. Emits kds:item_ready to expo view.
- PATCH /kds/tickets/:orderId/bump-all — Bump all items on a ticket. Emits kds:ticket_complete.

Tables

- GET /tables — All tables with real-time status for floor plan rendering.
- PATCH /tables/:id/status — Update status. Emits table:status_changed WebSocket event to all POS terminals.
- POST /tables/merge — Merge multiple tables into a combined order.
- POST /tables/split — Split a table order by seat into separate bills.

Inventory

- GET /inventory/ingredients — Stock list with current levels, par levels, and alert status.
- POST /inventory/stock-movements — Manual stock adjustment with reason code.
- POST /inventory/purchase-orders — Create PO. Triggers approval workflow if configured.
- PATCH /inventory/purchase-orders/:id/receive — Record received quantities. Auto-adjusts stock levels. Creates stock_movement records.
- POST /inventory/waste-logs — Log waste entry. Updates stock levels.
- POST /inventory/stock-count — Submit physical count. System calculates variance and creates adjustment movements.

Staff

- POST /staff — Admin creates staff account. Triggers onboarding email/SMS with setup link.
- GET /staff — List all staff with role, status, and current shift status.
- PATCH /staff/:id — Update role, status, permissions. Cannot demote the last admin.
- POST /staff/:id/deactivate — Revoke all active sessions. Prevent future login. Keeps historical records.

- GET /staff/schedule — Weekly rota view for branch.
- POST /staff/schedule/shifts — Create shift. Validates against availability and overtime rules.
- POST /attendance/clock-in — Record clock-in with method and device.
- POST /attendance/clock-out — Record clock-out. Calculates hours worked.

6. WebSocket Event Protocol

6.1 Connection & Rooms

- Client connects with auth token: socket.auth = { token: accessToken }
- Server middleware validates JWT on connection. Disconnects on invalid token.
- On successful connection, server joins socket to rooms: tenant:{tenant_id}, branch:{branch_id}, station:{prep_station} (kitchen clients), table:{table_id} (waiter clients)
- Each Socket.io namespace: /pos (FOH terminals), /kitchen (KDS screens), /manager (back-office and mobile app)

6.2 Events Reference

Event Name	Direction	Payload / Purpose
order:created	Server → POS + Kitchen	{ order_id, table_id, items[], order_type } — New order received
order:item_added	Server → Kitchen station	{ order_id, item, station } — New item sent to station
order:course_fired	Server → Kitchen	{ order_id, course_number, items[] } — Course fired by waiter
order:status_changed	Server → POS + Manager	{ order_id, old_status, new_status } — Order lifecycle change
order:ready	Server → POS (waiter)	{ order_id, table_id } — Full order ready for service
kds:item_bumped	Server → Expo + Manager	{ order_id, item_id, station, cook_time_seconds } — Item marked ready
kds:ticket_complete	Server → Expo + POS	{ order_id } — All items on ticket complete
table:status_changed	Server → All POS	{ table_id, old_status, new_status } — Floor plan update
menu:item_86d	Server → All POS + QR menu	{ item_id, name_fr, name_ar } — Item unavailable broadcast
menu:item_restored	Server → All POS + QR menu	{ item_id } — Item available again
inventory:low_stock	Server → Manager app	{ ingredient_id, name, current_stock, par_level } — Alert
payment:captured	Server → POS terminal	{ order_id, amount_cents, method } — Payment confirmed
reservation:arrived	Server → Host POS	{ reservation_id, customer_name, party_size, table_id }
staff:clocked_in	Server → Manager	{ user_id, name, shift_id, clocked_in_at }
kds:alert_overdue	Server → Kitchen + Manager	{ order_id, item_id, elapsed_seconds, target_seconds }

7. Frontend Architecture — Complete

7.1 App Structure

- The Flutter codebase targets three surfaces: POS Terminal App (Android tablet/dedicated POS hardware), Back-Office Web App (browser, responsive), Mobile Manager App (iOS + Android phone)
- Single monorepo with shared packages: core (models, API client, constants), ui (design system components), features (feature-by-feature modules)
- Feature-first folder structure: lib/features/{feature_name}/{data,domain,presentation}

7.2 Design System

- All UI components built from scratch in a shared ui package. No dependency on any third-party UI kit.
- Design tokens: color palette, typography scale, spacing grid (4px base), elevation, border radius — all defined as Dart constants
- Components include: RmsButton (primary, secondary, danger, ghost), RmsInput (with validation states), RmsCard, RmsModal, RmsBottomSheet, RmsTable, RmsStatusBadge, RmsCurrencyDisplay, RmsSearchBar, RmsDatePicker (RTL-aware), RmsTimePicker, RmsFloorPlanCanvas (custom painter), RmsKdsTicket, RmsReceiptPreview
- All interactive components have loading, disabled, error, and success states
- Tap targets minimum 48x48px per accessibility guidelines

⚠ *No component should hardcode LTR layout. Always use start/end alignment, Directionality, and TextDirection.*

7.3 Localization Architecture

- Two ARB files: app_fr.arb and app_ar.arb — one key per UI string
- All database-sourced user-facing strings (item names, category names, error messages from server) use the _fr or _ar field based on active locale
- Currency display: uses NumberFormat.currency(locale: activeLocale, symbol: tenant.currency_symbol)
- Date/time: uses DateFormat(locale: activeLocale). French: dd/MM/yyyy HH:mm. Arabic: uses Arabic calendar with Arabic-Indic digits (configurable per tenant).
- RTL layout: Directionality widget at root. All Row → start/end. All padding/margin uses EdgeInsetsDirectional. All icons that imply direction (arrows, menus) flip with Directionality.
- Locale preference stored per user in backend. Falls back to tenant default locale. Overridable per POS device (stored in Hive).
- Receipts and PDFs: generated in the locale of the order's customer profile. Falls back to tenant default.

7.4 POS Terminal App — Screen Map

- Login Screen: Email+password OR PIN entry. PIN entry is a large-button numpad for quick touchscreen use.

- Floor Plan Screen: Interactive canvas built with CustomPainter. Tables rendered as colored shapes. Real-time status via WebSocket. Tap to open table.
- Order Screen: Left panel = item selection (categories + items). Right panel = current order, items with modifiers, seat assignments, totals. Top bar: table info, covers count, timer, waiter name.
- Item Detail Modal: Full-screen. Modifier groups rendered as chips (single: radio behavior, multi: checkbox behavior). Allergen warnings shown inline.
- Payment Screen: Amount due, multiple tender methods, split bill by seat or percentage, change calculation for cash.
- Receipt Screen: On-screen preview with print button and digital send (email/SMS) option.
- KDS Screen (tablet, landscape): List of active tickets ordered by time. Color coding: green (< 50% of target time), yellow (50-90%), red (> target). Bump button per item and per ticket. Station filter tabs at top.
- 86 Management Screen: Grid of all items. One tap toggle. Changes broadcast instantly.
- Reservation Arrival Screen: Today's bookings list, status filter, arrive/no-show actions.

7.5 Back-Office Web App — Screen Map

- Dashboard: Real-time KPI tiles, revenue sparkline (today vs yesterday vs last week), covers chart, top items today, low stock alerts, pending reservations.
- Menu Management: Category tree on left, item list on right. Drag to reorder. Inline edit or full edit modal. Photo upload with crop. Modifier group management. Recipe attachment.
- Inventory: Ingredient list with stock levels (color-coded by par). Stock movement log. Waste log entry. Supplier directory. PO creation and tracking board.
- Staff Management: Staff list with role badges and shift status. Add staff form (Admin only). Role and permission editor. Schedule weekly view with shift creation. Attendance log per employee.
- Reports: Date range picker, report type selector, filterable data table, chart visualization, CSV/PDF export.
- CRM: Customer search, profile viewer, loyalty tier management, campaign creation wizard (audience → content → schedule → review).
- Finance: Cash reconciliation form, invoice inbox, P&L viewer, tax summary, accounting export.
- Settings: Restaurant profile, branch management, table layout editor, tax rates, payment methods, notification templates (fr + ar), integrations, API key management.

7.6 Offline Mode

- POS terminal monitors connectivity via connectivity_plus and backend heartbeat ping every 30 seconds.
- On connectivity loss: UI shows offline banner. POS continues operating using Hive-cached menu snapshot.
- Orders created offline stored in Hive with status: pending_sync.
- Payments in offline mode: cash only. Card payments are blocked with a visible message.
- On reconnect: sync queue processes pending_sync orders in order. Conflict resolution: server is authoritative for table status.
- Menu updates received while offline queued and applied on reconnect.
- Offline mode event displayed in audit log with is_offline: true flag.

8. Feature Modules — Production Specification

8.1 Point of Sale & Order Management

Table Management

- Floor plan is stored as JSON in the database (tables with x, y, width, height, shape).
- Admin can edit the floor plan via a drag-and-drop canvas editor in back-office. Changes saved immediately.
- Real-time table status pushed via WebSocket to all connected POS terminals in the same branch.
- Table occupancy timer: starts when status transitions to occupied. Alert threshold configurable per tenant (default: 90 minutes). Timer displayed on floor plan card.
- Merge tables: creates a logical merge record linking multiple table IDs to one primary order. All tables show as occupied. Unmerge splits back to individual bills.
- Table transfer: reassign an open order from one table to another without closing.

Order Flow

- Order creation validates: table is available (or delivery/takeaway), at least one item added, waiter has permission.
- Every order state transition is recorded with timestamp, user_id, and previous state in audit_log.
- Special requests and allergen alerts: any item flagged as containing an allergen category from the customer's dietary profile triggers an inline warning that the waiter must acknowledge before adding.
- Course management: items assigned to courses 1-n. Waiter fires a course from POS → order_items for that course transition to fired status → WebSocket event to kitchen station.

Payment Processing

- Payment session created when waiter taps Pay. Session tracks: total due, amount tendered so far, remaining balance.
- Multi-tender: waiter can accept multiple payment methods against one order. Session closes when remaining balance reaches zero.
- Cash: system calculates and displays change due. Drawer open command sent to cash drawer via ESC/POS.
- Card payments: via payment gateway SDK. Terminal shows card prompt. On gateway callback: payment record created, order closed. Handles network timeout with idempotent retry.
- Refunds: partial or full. Requires manager authorization code. Creates negative payment record. If card: gateway refund initiated. Updates order status.
- Voids: only on orders not yet paid. Requires manager authorization. Restores table to available. Notifies kitchen to discard tickets.
- No tip fields, tip prompts, tip calculations, or tip-related logic exist anywhere in the payment flow.

Discounts

- Discount rules stored in database: name, type (percentage/fixed/bogo/category_pct/happy_hour), value, conditions_json (time range, minimum order, applicable categories), requires_manager_approval (bool), max_uses, expiry.
- Happy hour rules: system checks current time against rule time range on every order. Auto-applies if conditions met.
- Manual discount: cashier selects discount type and value. If requires_manager_approval is true, PIN prompt for manager. After authorization, discount logged with manager_id.

- Coupon codes: validated against discount_codes table. Single-use codes marked redeemed on use. Redemption logged with order_id.

8.2 Kitchen Operations

KDS Full Specification

- KDS app is the Flutter app running on kitchen tablets (Android, landscape mode) connected to the same WebSocket namespace as POS.
- Each KDS instance configured with a prep_station. It only receives tickets for items routed to its station.
- Ticket card shows: order number, table number, covers, time elapsed (live counter), list of items with modifiers and notes, seat numbers, allergen warnings in red.
- Bump item: marks order_item.status = ready. Emits kds:item_bumped. If all items on an order for this station are ready, emits a station-complete notification to expo.
- Expo view: shows consolidated view of all tickets. An item card shows all stations and their completion status. Expo bumps the order to served when all stations are complete.
- Overdue alert: BullMQ delayed job created when item is fired. If item not bumped within target_cook_time, emits kds:alert_overdue WebSocket event. KDS ticket card turns red.
- Recall: last 10 bumped tickets accessible via Recall button in case of bump error.

Food Safety & HACCP

- Temperature log form: ingredient/equipment name, recorded temperature, target range (from config), pass/fail status, notes, timestamp, user. Required fields validated.
- IoT integration: temperature readings can be ingested via POST /iot/temperature-readings with device API key. Auto-creates log entry. Alert if out of range.
- Daily checklists: opening/mid-service/closing. Configurable per tenant. Each checklist item has a required field (must be checked before shift can proceed) flag.
- Compliance export: admin can export HACCP log PDF for any date range. Generated by BullMQ worker using Puppeteer.

8.3 Inventory & Supply Chain — Full

Bill of Materials (BOM) Auto-Deduction

- When an order_item status transitions to served (or on order close if configured), the system reads the item's recipe ingredients and creates stock_movement records with movement_type = sale_deduction for each ingredient.
- BOM deduction is processed by a BullMQ job (not synchronously on the order close path) to avoid adding latency to the payment endpoint.
- If an ingredient stock level drops below par_level after deduction, an inventory:low_stock event is emitted and a notification queued.

Purchase Order Workflow

- States: draft → pending_approval → approved → sent → partial_received → received → cancelled
- Approval workflow: if PO total exceeds tenant-configured threshold, requires approval from a user with inventory.po.approve permission before it can be sent.

- On approval, system generates a PDF PO and emails it to the supplier (via BullMQ job).
- Receiving: clerk scans or enters received quantities per line. System creates stock_movement records. Discrepancies flagged and stored. Invoice matching happens in finance module.

Stock Count

- Admin initiates a count session for a branch. Session freezes expected quantities at count start time.
- Count submitted via mobile app (barcode scan to find ingredient, enter physical count).
- On count completion, system calculates variance per ingredient: variance = actual - expected. Variance report generated. Adjustment movements created for non-zero variances.

8.4 Staff & HR — Full

Account Creation (Admin-Only Flow)

- Admin opens Add Staff form: first_name (fr+ar), last_name (fr+ar), email, phone, role, branch, preferred_locale, PIN (4-6 digits).
- System validates email uniqueness within tenant. Hashes PIN with bcrypt.
- Sends onboarding email (in staff's preferred locale) with a one-time setup token (valid 48 hours) for password creation.
- Staff completes password setup via POST /auth/setup-password with token + new password. Token invalidated after use.
- Admin can resend onboarding link if token expired. Admin can set a temporary password and force change on first login.
- Deactivation: sets is_active = false, revokes all active tokens in Redis, prevents login. Historical data preserved.

Payroll Export

- System calculates worked hours per staff per period from attendance records.
- Overtime flagged based on configurable weekly hour threshold.
- No tip calculations. Payroll export is hours-only.
- CSV export: columns = employee_name, role, period_start, period_end, regular_hours, overtime_hours, days_worked.
- Export sent to admin email or downloadable from back-office.

8.5 Analytics & Reporting — Full

Report Types

- Sales Summary: revenue, covers, avg check, per-category and per-item breakdown. Filterable by branch, waiter, order type, date range.
- Hourly Heatmap: covers and revenue plotted hour by hour for each day of the week. Shows peak times.
- Menu Engineering Matrix: calculated per item: popularity_score = item_count / avg_item_count. margin_score = item_margin / avg_margin. Plotted in Star/Plow/Puzzle/Dog quadrant.
- Food Cost Report: theoretical cost (BOM * sales) vs actual cost (purchase invoices). Variance percentage. Per-category breakdown.

- Inventory Valuation: current stock * cost_per_unit. Valued at last purchase price.
- Staff Hours Report: worked hours vs scheduled hours per employee per period.
- Void & Refund Report: all voids and refunds with authorizing manager, reason, and amount.
- Discount Usage Report: redemptions per discount rule, total discount given, coupon redemption rates.
- Reservation Report: total bookings, shows vs no-shows, conversion rate, average party size.
- Customer Feedback Report: NPS score trend, rating distribution by category, verbatim responses with sentiment flags.

Dashboard KPI Caching Strategy

- Dashboard KPIs recalculated every 30 seconds via a scheduled BullMQ job per tenant.
- KPI values stored in Redis with key: kpi:{tenant_id}:{branch_id}:{date}. TTL: 60 seconds.
- Dashboard GET /analytics/dashboard returns cached KPIs. If cache miss, calculates synchronously and caches.
- Historical data (reports for past dates) computed on demand, cached with 1-hour TTL.

8.6 CRM & Loyalty — Full

Customer Auto-Building

- Customer record created automatically when: a reservation is made (phone/email captured), a loyalty card signup occurs, or admin manually creates from back-office.
- Guest matching: on order close, if customer_id is linked to the order, loyalty points are awarded automatically.
- Visit history auto-updated: total_visits incremented, last_visit_at set, lifetime_spend_cents updated.

Loyalty Engine

- Points rules configurable per tenant: points_per_currency_unit (e.g., 1 point per 10 DZD), flat_points_per_visit, bonus points on birthday month.
- Tier evaluation: on every loyalty transaction, system checks if customer's total points qualify for a higher tier. Tier upgrade notification sent automatically.
- Redemption: at POS payment, cashier looks up customer by phone/QR. System shows points balance and available rewards. Reward applied as a discount amount. Loyalty transaction created with negative points_delta.

Marketing Campaigns

- Audience segmentation: filter by tier, visit_count (> / < n), last_visit_at (> n days ago), birthday_month, lifetime_spend_cents range.
- Content: subject, body_fr, body_ar. Template variables: {{customer_name_fr}}, {{customer_name_ar}}, {{loyalty_points}}, {{restaurant_name_fr}}, {{restaurant_name_ar}}.
- Schedule: send immediately or schedule for future datetime. Recurring campaigns (e.g., every birthday month) supported.
- Delivery: BullMQ processes the send queue with rate limiting (max 50 SMS/min, 100 email/min) to avoid provider throttling.
- Analytics: open_rate (email), delivery_rate, redemption_rate (if campaign includes a coupon code).

8.7 Finance & Accounting — Full

Cash Reconciliation

- End-of-day flow: cashier counts physical cash in drawer → enters total → system calculates expected cash (opening float + cash sales - cash refunds) → shows variance → manager signs off.
- Z-report: final end-of-day report per register. Shows all payment method totals, discount totals, tax collected, void totals. Printable and emailed to admin.
- X-report: mid-day snapshot without closing the drawer. Can be run any number of times.

Tax Engine

- Tax rates configured per tenant with type (inclusive/exclusive), percentage, and applicable item categories.
- On order close, system calculates tax breakdown per rate. Stored in order record as tax_breakdown_json.
- VAT/TVA summary report: total taxable sales, tax collected, by rate — ready for tax authority submission.
- Fiscal printer: where required by law (e.g., Algeria), system sends fiscal data to fiscal printer via serial/USB driver. Fiscal receipt number stored on order record.

9. Security — Production Requirements

9.1 Transport & Protocol

- HTTPS enforced everywhere. TLS 1.2 minimum, TLS 1.3 preferred. HTTP requests redirected to HTTPS.
- WebSocket connections over WSS only.
- HSTS header enforced (max-age 31536000, includeSubDomains).
- Content Security Policy header on all web responses.
- CORS: explicit allowlist of tenant domains. No wildcard (*) origins in production.

9.2 Authentication Security

- Passwords: bcrypt with cost factor 12. Minimum 8 characters, validated server-side.
- PINs: bcrypt-hashed. 4-6 digits. Cannot be the same as the user's numeric password suffix.
- JWT: signed with RS256 (RSA asymmetric). Private key stored in secrets manager. Public key rotatable without downtime.
- Access token lifetime: 15 minutes. Contains: sub (user_id), tenant_id, role, permissions[], branch_id, jti (unique token ID).
- Refresh token: 7-day lifetime. Stored as hash in Redis with key refresh:{jti}. Rotated on every use (old token invalidated immediately). Sliding window: each refresh extends by 7 days.
- Refresh token bound to user-agent + IP. Mismatch triggers revocation and admin alert.
- Brute-force protection: rate limit 5 failed login attempts per IP per 15 minutes. Account lock after 10 failed attempts with admin notification.
- Admin can revoke all sessions for a user: deletes all Redis keys matching refresh:{user_id}:*.

9.3 Authorization

- Every API route declares required permission string (e.g., orders.create, reports.view, staff.manage).
- requirePermission middleware checks JWT permissions[] claim. Returns 403 if missing.
- Tenant isolation middleware: on every request, validates that the resource's tenant_id matches the JWT's tenant_id. This runs after auth, before business logic.
- Super Admin routes: separate namespace /super-admin/. Only accessible with super_admin role flag in JWT. Not part of tenant JWT.
- No security by obscurity: endpoints not included in JWT permissions return 403, not 404.

9.4 Data Security

- All user-generated inputs sanitized server-side via fastify-sanitize-html for text fields.
- All database queries use parameterized statements via Knex. No string concatenation in SQL.
- File uploads: MIME type validated server-side (not just extension). Max file size enforced per type (photos: 5MB, documents: 20MB). Stored in S3 with server-side encryption (AES-256).
- Card data: never stored on RMS systems. Payment tokenized by gateway before arriving at RMS API.

- PII at rest: customer email, phone encrypted at application level (AES-256-GCM) with key per tenant stored in secrets manager.
- GDPR/privacy: DELETE /customers/:id anonymizes personal data (replaces with anonymized_customer_{id}) and deletes name, email, phone, birthday. Order history preserved with anonymized customer_id for financial records.

9.5 Audit & Compliance

- Every write operation (create, update, delete, status change) creates an audit_log entry: user_id, tenant_id, action, entity_type, entity_id, old_value_json, new_value_json, ip_address, user_agent, timestamp.
- Audit log is append-only. No UPDATE or DELETE on audit_log table. Access controlled to admin and super admin only.
- Audit log retention: 2 years minimum. Archivable to S3 Glacier for long-term.
- All payment operations additionally logged to a payments_audit table with gateway transaction IDs.

10. Performance & Scalability

10.1 API Performance Targets

Endpoint Category	P95 Target	P99 Target
POS critical (order create/update, payment, table status)	< 100ms	< 200ms
Menu fetch (Redis-cached)	< 30ms	< 60ms
KDS ticket list	< 80ms	< 150ms
Dashboard KPIs (Redis-cached)	< 50ms	< 100ms
Report generation (historical)	< 2s	< 5s
PDF generation (Puppeteer, async)	< 10s	< 20s
WebSocket event propagation	< 200ms	< 500ms
Search (menu items, customers)	< 100ms	< 200ms

10.2 Database Performance

- All foreign keys have corresponding indexes. All WHERE clause fields on hot queries have indexes.
- Critical composite indexes: (tenant_id, status) on orders; (tenant_id, is_available) on menu_items; (tenant_id, user_id, created_at) on attendance; (tenant_id, created_at) on stock_movements.
- EXPLAIN ANALYZE run on every new query added to a critical path. Any query with a seq scan on a large table is rejected.
- Large tables (orders, stock_movements, audit_log) partitioned by tenant_id and date range (monthly partitions). Older partitions archived and detached from main table.
- Read replicas: analytics and report queries routed to a read replica. Write operations go to primary.
- Connection pooling: PgBouncer in transaction mode. Max 20 connections to PostgreSQL per backend instance. Backend can scale horizontally without exhausting DB connections.

10.3 Caching Strategy

Cache Key Pattern	TTL	Invalidation Trigger
menu:{tenant_id}:{branch_id}	24 hours	Any menu write (item, category, modifier)
kpi:{tenant_id}:{branch_id}:{date}	60 seconds	Auto-expiry + BullMQ 30s refresh job
table_status:{tenant_id}:{branch_id}	No TTL (pub/sub)	Every table status change event
session:{jti}	15 minutes	Logout or token revocation

Cache Key Pattern	TTL	Invalidation Trigger
refresh:{jti}	7 days	Use, logout, or admin revocation
rate_limit:{ip}:{route}	15 minutes	Auto-expiry
report:{hash_of_params}	1 hour	Manual cache bust on data change
loyalty_tier:{tenant_id}	1 hour	Tier config update

10.4 Scalability Design

- Backend is fully stateless. All shared state lives in Redis. Any number of backend instances can serve any request.
- Socket.io uses Redis adapter (@socket.io/redis-adapter) so WebSocket events emitted by one instance are received by clients connected to other instances.
- BullMQ workers run as separate processes/containers from the API server. Can scale independently. Queues: notifications, pdf_generation, report_generation, email_campaigns, bom_deduction, po_email.
- Horizontal scaling: add backend API instances behind Nginx load balancer. Nginx sticky sessions NOT required (stateless backend).
- Vertical scaling: database scales to the PostgreSQL instance size. Horizontal read scaling via read replicas.
- CDN: all static Flutter web assets served via Cloudflare CDN with far-future cache headers.
- Image optimization: menu item photos resized and WebP-converted on upload via a BullMQ job. Multiple sizes stored: thumbnail (100x100), card (400x400), full (800x800).

11. SaaS Platform Layer

11.1 Super Admin Portal

- Accessible at: https://platform.rms.com (separate app, not part of tenant app)
- Super Admin can: create tenant accounts, assign subscription plan, suspend/reactivate tenants, view platform-wide metrics (total tenants, MRR, active sessions), manage subscription plan definitions.
- Tenant creation flow: Super Admin enters restaurant name, admin email, plan tier, country → system creates tenant record, creates admin user, sends admin onboarding email.
- Platform dashboard: tenant growth chart, MRR chart, churned vs new tenants, top tenants by order volume.

11.2 Subscription & Billing

- Subscription plans defined in DB: Starter (1 branch, 10 staff, basic features), Pro (3 branches, 50 staff, analytics, CRM), Enterprise (unlimited, all features, API access, white-label).
- Billing: Stripe integration. Tenant payment method stored in Stripe (not in RMS DB). Invoices generated by Stripe.
- Feature gating: middleware checks tenant.plan_tier against feature requirements. Returns 403 with upgrade prompt if feature not available on current plan.
- Grace period: 14 days after subscription expiry before account is suspended. Admin notified at expiry, 7 days, and 1 day before suspension.
- Suspension: all API calls return 402 Payment Required. Admin can access billing page only to renew.

11.3 Tenant Onboarding Flow

Full automated onboarding sequence after Super Admin creates tenant:

- 1. Tenant record created with status = onboarding.
- 2. Admin user record created with is_active = true, onboarding_token generated.
- 3. Onboarding email sent to admin email in French (default) with: platform URL, setup password link, getting started guide link.
- 4. Admin sets password, lands on onboarding wizard.
- 5. Onboarding wizard: Step 1 = Restaurant profile (name fr/ar, logo, address, phone, timezone, currency). Step 2 = First branch setup. Step 3 = Tax rates. Step 4 = Menu import (CSV template or manual). Step 5 = Staff accounts. Step 6 = Table layout.
- 6. On wizard completion, tenant.status transitions to active.
- 7. 30-day free trial begins. Stripe trial subscription created.

12. Infrastructure & Deployment

12.1 Container Architecture

```
services:
  api:          # Fastify API server. Replicas: 2 minimum in prod.
  worker:       # BullMQ worker. Replicas: 2. Processes: notifications, pdf, reports,
campaigns.
  postgres:     # PostgreSQL 16. Single primary + 1 read replica (prod).
  redis:        # Redis 7. Used for cache, sessions, queues, pub/sub.
  pgbouncer:    # Connection pooler in front of PostgreSQL.
  nginx:        # Reverse proxy, SSL termination, load balancing.
  flutter-web:  # Flutter web build served via Nginx static file server.
  meilisearch:  # Optional: search service for menu items and customers.
```

12.2 Environment Configuration

- Three environments: development (local Docker Compose), staging (Railway/Fly.io, mirrors prod), production (Railway/Fly.io or Kubernetes)
- Environment variables: DATABASE_URL, REDIS_URL, JWT_PRIVATE_KEY, JWT_PUBLIC_KEY, S3_BUCKET, S3_ENDPOINT, S3_ACCESS_KEY, S3_SECRET_KEY, SENDGRID_API_KEY, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, SENTRY_DSN, NODE_ENV, PORT, LOG_LEVEL
- Secrets never in source code. Never in Docker images. Injected at runtime via Doppler or platform env config.

12.3 CI/CD Pipeline

- On pull request: lint (ESLint + Prettier), unit tests (Vitest), integration tests (test DB), Flutter analyze, Flutter tests.
- On merge to main: build Docker image, push to container registry (GHCR), deploy to staging, run smoke tests, manual approval gate, deploy to production.
- Zero-downtime deployments: rolling update strategy. Old containers drained before shutdown.
- Database migrations: run automatically on deploy before new API version starts. Migrations must be backward-compatible with the previous API version to support rolling updates.
- Rollback: previous Docker image tag retained. Rollback by retagging and redeploying. DB migrations must be reversible.

12.4 Observability

- Structured JSON logs via Pino. Log levels: error, warn, info, debug. Each log line includes: timestamp, level, request_id (UUID per request), tenant_id, user_id, route, duration_ms.
- Request tracing: X-Request-ID header propagated through all internal service calls and async jobs.
- Metrics: custom Prometheus metrics exported at /metrics endpoint (not publicly accessible). Key metrics: http_request_duration_ms, active_websocket_connections, queue_job_duration_ms, queue_job_failures, cache_hit_ratio, db_query_duration_ms.

- Alerting: Grafana alerts on: P95 latency > 500ms for 5 minutes, error rate > 1% for 2 minutes, queue depth > 1000 jobs, DB connection pool exhaustion, disk > 80%.
- Error tracking: Sentry captures unhandled errors with full stack trace, request context, user ID, tenant ID. PII stripped before sending to Sentry.
- Uptime monitoring: external synthetic checks on /health endpoint every 60 seconds. Alert on 2 consecutive failures.
- Health check endpoint: GET /health returns { status: ok, db: ok|error, redis: ok|error, version: string, uptime_seconds: number }

13. Hardware Integrations

13.1 Receipt Printers

- Protocol: ESC/POS — industry standard for thermal printers.
- Supported: Epson TM series, Star TSP series, any ESC/POS-compatible USB/network printer.
- Flutter package: flutter_thermal_printer or direct TCP socket to printer IP.
- Receipt layout: configurable via tenant_config (logo, header text, footer text, show/hide fields). Rendered in active locale (fr or ar). Arabic receipt uses RTL text alignment.
- Receipts include: QR code linking to digital receipt URL, order number, itemized list, subtotal, tax breakdown, total, payment method, timestamp, server name.
- Kitchen printer: separate printer at kitchen pass for restaurants not using KDS screens. Kitchen ticket format: table, covers, items with modifiers, notes, time. No prices on kitchen tickets.

13.2 Cash Drawers

- Connected to receipt printer via RJ11 cable. Drawer open command sent as ESC/POS command after cash payment captured.
- Configurable: some tenants require manager PIN to open drawer outside of a payment. Configurable in tenant_config.

13.3 Barcode Scanners

- Used in: inventory counting (scan ingredient barcode), item lookup at POS (scan product barcode to add to order).
- HID keyboard emulation scanners (most common): work natively as keyboard input. Flutter app listens for rapid-fire key input ending in Enter to detect scan events.
- Camera-based scanning: mobile_scanner Flutter package for smartphone/tablet camera barcode reading.

13.4 Card Payment Terminals

- Integration via payment gateway SDK (Stripe Terminal, HiPay, CMI, or local acquirer SDK).
- Flow: RMS creates PaymentIntent via gateway API → sends to terminal → terminal handles card interaction → gateway webhooks payment status to RMS → RMS confirms order payment.
- Idempotency: PaymentIntent ID stored before sending. If network drop occurs, RMS polls gateway for status on reconnect.
- Supported terminal connectivity: USB, Bluetooth, or LAN (gateway-dependent).

13.5 Customer-Facing Display

- Second screen connected to POS terminal showing order items and totals as cashier adds them.

- Implemented as a second Flutter web view pushed to a secondary display (Android Display Manager API or secondary HDMI screen on desktop).
- Shows: item list, subtotal, tax, total. Updates in real-time as items are added.
- Idle state: tenant logo and promotional message (configurable).

14. External Integrations

14.1 Delivery Aggregators

- Integration-ready webhook receiver: POST /integrations/aggregators/{provider}/orders accepts inbound orders from aggregator webhooks (Uber Eats, Glovo, etc.).
- Aggregator orders mapped to internal order format. order_type = delivery. Source field stores aggregator name and external order ID.
- Status sync: when order status changes in RMS, outbound webhook sent to aggregator API to update status on their platform.
- Menu sync: admin can push menu changes to configured aggregators via PUT /integrations/aggregators/{provider}/menu.

14.2 Accounting Exports

- Daily journal export: JSON or CSV file per day with: total revenue by category, total COGS, total tax collected, total discounts, total refunds. Compatible with QuickBooks, Xero, and Sage import formats.
- Export triggered manually from finance module or on a schedule (auto-send to accounting system email at end of day).

14.3 Notification Providers

- Email: SendGrid or Resend. Templates stored in DB (both fr and ar versions). Dynamic variables substituted server-side before sending.
- SMS: Twilio (international) or Africa's Talking (North Africa/sub-Saharan). Provider configurable per tenant by region.
- Push: Firebase Cloud Messaging for mobile app push notifications. Device tokens stored per user.
- All notification dispatches logged in notifications_log table: recipient, channel, template, status, provider_message_id, sent_at.

14.4 Payment Gateways

- Stripe: international cards, Apple Pay, Google Pay.
- HiPay: regional preference for North Africa/MENA.
- CMI (Centre Monétique Interbancaire): Moroccan market.
- Gateway is configurable per tenant. API keys stored per tenant in encrypted tenant_config.
- All gateway callbacks verified via webhook signature before processing.

15. Testing Requirements

15.1 Backend Testing

- Unit tests: all service-layer business logic (order state machine, discount rule engine, loyalty points calculation, BOM deduction, stock level calculations) tested in isolation with mocked DB.
- Integration tests: all API endpoints tested against a real PostgreSQL test database (Docker-based). Test tenant and test users seeded before each test suite.
- Coverage target: 80% line coverage minimum on all service modules.
- Load testing: k6 scripts for critical paths. Order creation: 500 RPS sustained for 60 seconds. Menu fetch: 2000 RPS. Payment processing: 200 RPS. All must meet P95 targets.
- Security testing: OWASP ZAP scan run in CI against staging. SQL injection fuzz tests on all input fields.

15.2 Frontend Testing

- Widget tests: all design system components tested for correct rendering in LTR (fr) and RTL (ar) modes.
- Golden tests: visual snapshot tests for KDS ticket, receipt preview, floor plan, and dashboard widgets.
- Integration tests: Flutter integration_test for critical flows: order creation, payment, KDS bump, staff account creation.
- Accessibility: all screens audited for minimum tap target size, sufficient color contrast, and screen reader compatibility.

15.3 End-to-End Testing

- Patrol (Flutter E2E) tests covering: full order lifecycle from table selection to order close, offline order creation and sync on reconnect, language switch and RTL layout verification, admin staff creation and first login flow.
- E2E tests run against staging environment in CI before production deploy.

16. Explicitly Excluded Features

The following are definitively out of scope and must not be built or referenced:

Excluded Feature	Reason / Note
Tip management (collection, distribution, tracking, display)	Business decision. No tip UI, no tip DB column, no tip calculation anywhere.
Staff self-registration	Security requirement. Admin-only account creation.
Cross-tenant data access or shared menus between tenants	Core SaaS isolation requirement.
Built-in payroll processing or salary payment	Export-only. Integration with external payroll system.
Franchise royalty management	Out of scope for this version.
Public multi-restaurant marketplace or storefront	Each tenant has their own isolated system.
Cryptocurrency payments	Not in payment gateway scope.
Table-side entertainment or digital signage beyond customer display	Out of scope.
Catering / off-site event management	Out of scope.
Employee performance review workflows (beyond manager notes)	Out of scope for this version.

17. Production Readiness Checklist

Every item on this list must be completed and verified before launch:

Security

- HTTPS enforced on all endpoints and WebSocket connections
- JWT RS256 signed with private key in secrets manager
- Rate limiting active on all routes
- Tenant isolation verified by automated test
- OWASP ZAP scan passed with no high or critical findings
- PII encryption verified in database
- Audit log capturing all write operations
- Admin session revocation tested end-to-end

Performance

- P95 API latency < 200ms on POS-critical endpoints under load
- WebSocket event propagation < 500ms verified with load test
- Database indexes verified with EXPLAIN ANALYZE on all hot queries
- Redis cache hit ratio > 80% for menu and dashboard endpoints
- BullMQ worker queues processing without backlog under normal load

Reliability

- Offline mode tested: create order, lose internet, reconnect, verify sync
- Zero-downtime deployment tested on staging
- Database backup automated and recovery tested
- Health check endpoint returning accurate DB and Redis status
- Sentry error tracking capturing errors in production
- Grafana alerts configured and tested

Localization

- All UI strings translated in both app_fr.arb and app_ar.arb — zero missing keys
- RTL layout verified on all screens with Arabic locale active
- Receipts generated correctly in both French and Arabic
- Date, time, and currency formatted correctly for both locales
- Email and SMS notification templates available in both languages

SaaS Platform

- Tenant isolation E2E test: two tenants cannot access each other's data
- Subscription plan feature gating tested for each plan tier
- Tenant onboarding wizard completed in full test run
- Super Admin portal accessible and functional
- Stripe billing webhooks verified with Stripe CLI

Hardware

- ESC/POS receipt printing tested with Epson and Star printers
- Cash drawer open command tested
- Barcode scanner input detected correctly on POS order screen
- KDS display tested on Android tablet in landscape mode

END OF PRODUCTION SPECIFICATION

This document defines the complete production system. Build to this spec exactly.